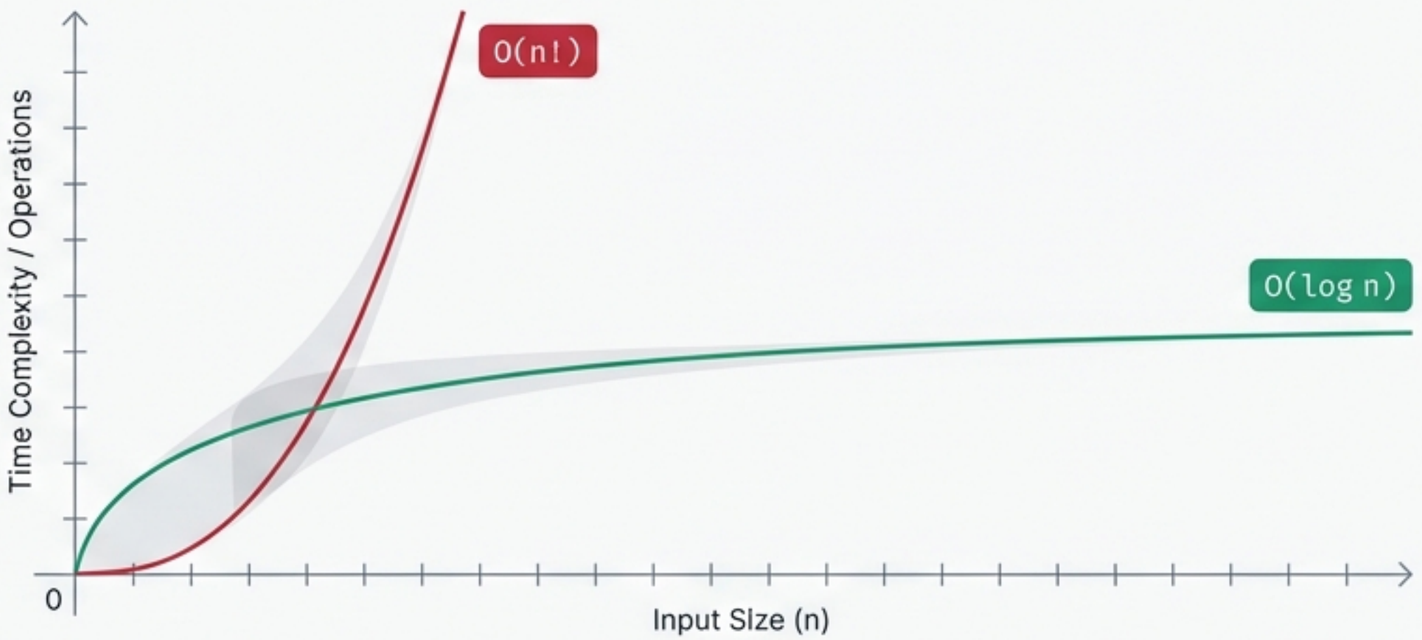
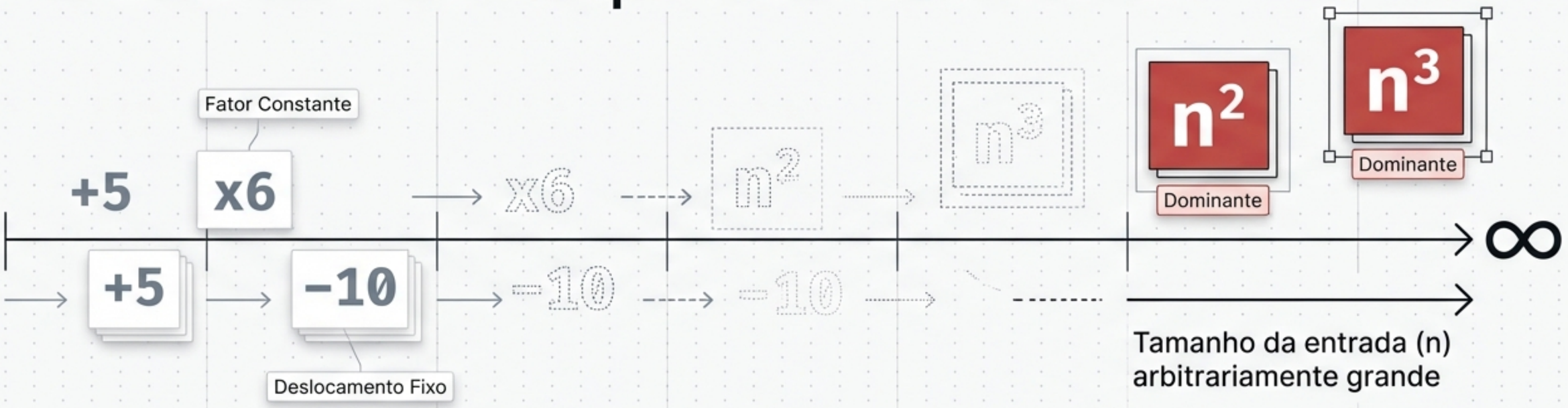
Notação Big O: A Matemática da Escalabilidade

> Desconstruindo a complexidade de algoritmos_



NotebookLM

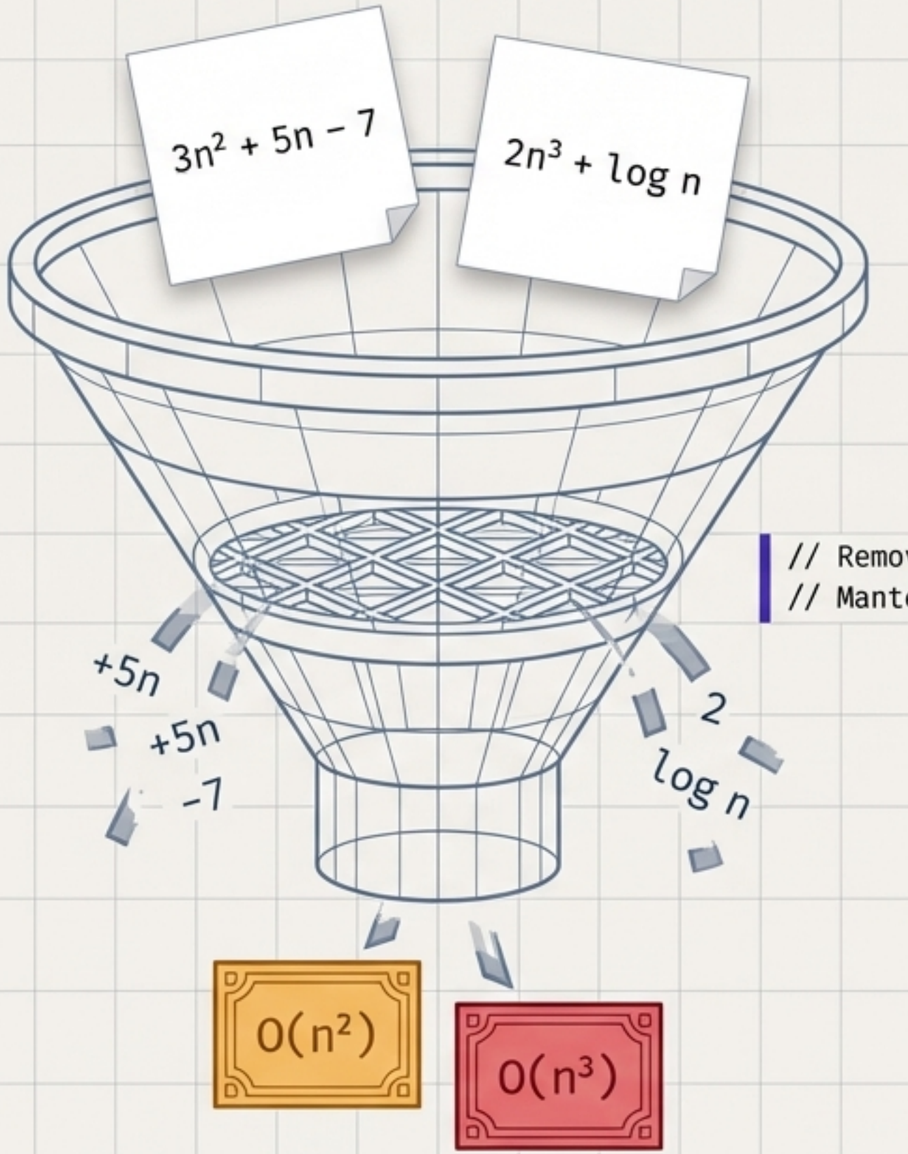
Constantes não importam no infinito.



O comportamento assintótico define o **algoritmo**. O tempo e o espaço são contabilizaddos em número de **passos**, não em segundos.

NotebookLM

Filtrando o Ruído




```
// Removendo constantes aditivas e multiplicativas
// Mantendo apenas o termo de maior grau
```

NotebookLM

O Duelo de Algoritmos

Algoritmo A

Tempo de execução:

$T_A(n)$

Qual o melhor?

Algoritmo B

Tempo de execução:

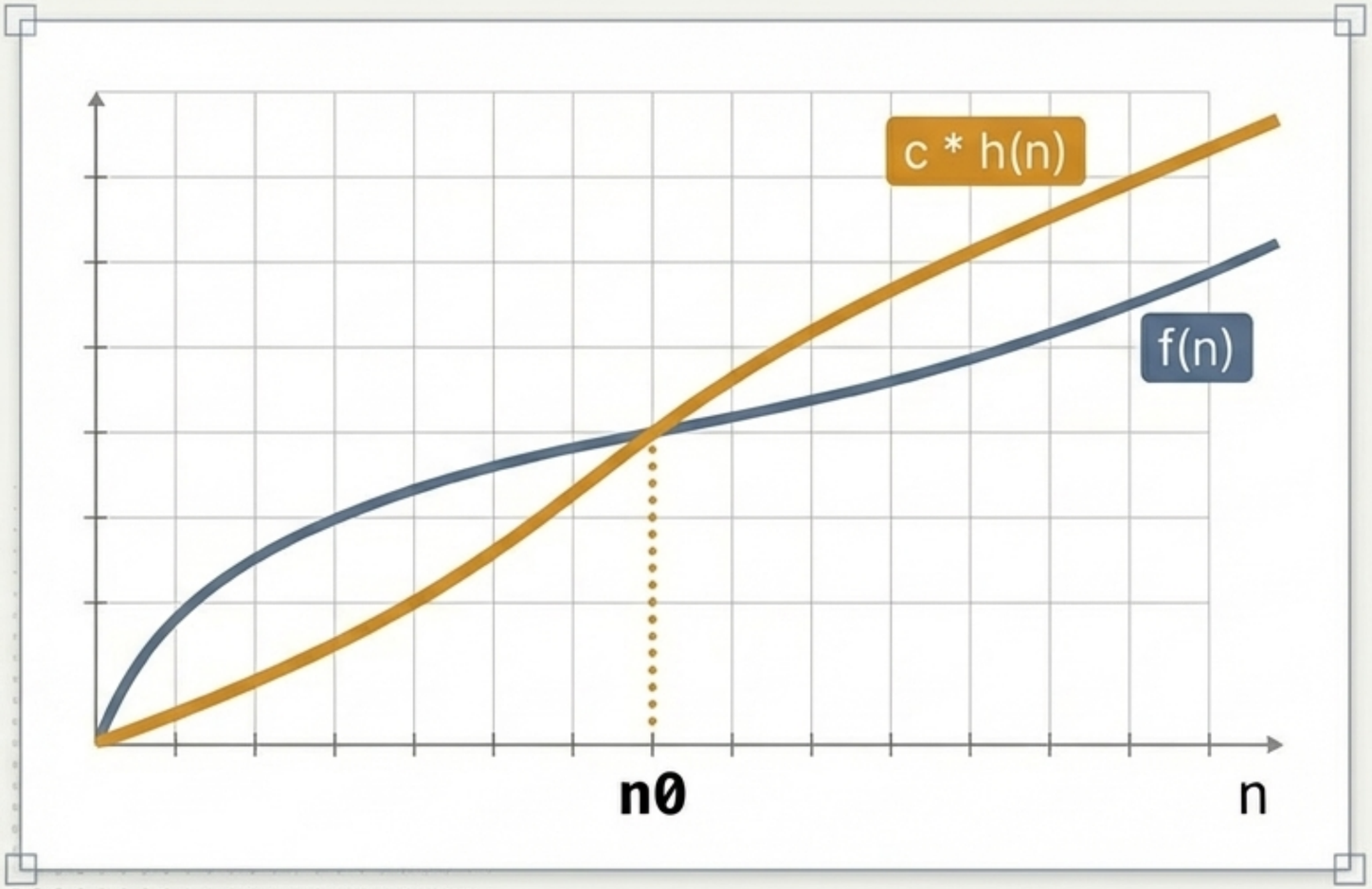
$T_B(n)$

Para resolver o mesmo problema P, a escolha depende do comportamento assintótico caracterizado pela notação O, não do hardware atual.

NotebookLM

A Definição Formal

$$f(n) = O(h(n))$$



Limite superior assintótico:
Existe uma constante c > 0
e um inteiro n0 tal que:

O(1) O(n) O(n²)

```
if (n > n0) {
  f(n) ≤ c * h(n)
}
```

NotebookLM

A Prova na Prática

O algoritmo $f(n) = 8n + 128$ é $O(n^2)$?

A Premissa

$$f(n) \leq c.n^2 \text{ ? Assume constante } c = 1.$$

A Substituição

$$8n + 128 \leq n^2$$

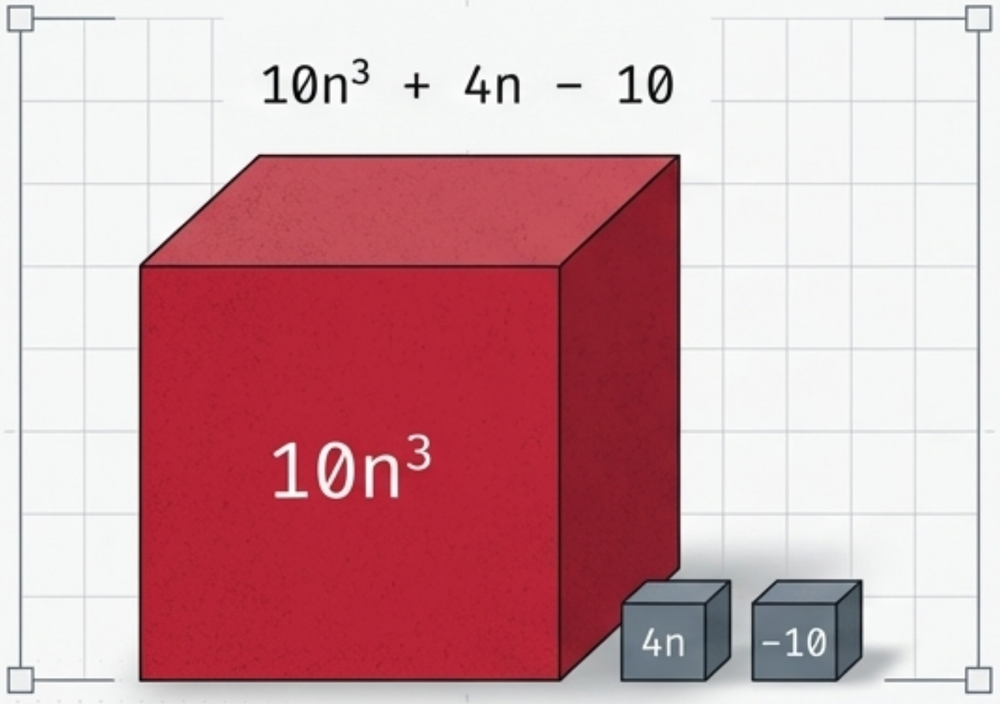
A Fatoração

$$0 \leq n^2 - 8n - 128 \longrightarrow 0 \leq (n - 16)(n + 8)$$

$n0 = 16$. Provado! Para todo $n \geq 16$, $8n + 128 \leq n^2$.

NotebookLM

O Comportamento Dominante



À medida que n aumenta, o termo cúbico começa a dominar completamente. A constante do termo cúbico torna-se irrelevante.

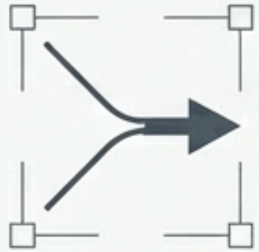
$f = n^2 - 1$	$O(n^2)$
$f = 403$	$O(1)$
$f = 5 \cdot 2^n + 5n^{10}$	$O(2^n)$
$f = 5 + 2 \log n + 3 \log^2 n$	$O(\log^2 n)$

NotebookLM

O Manual de Regras Big O

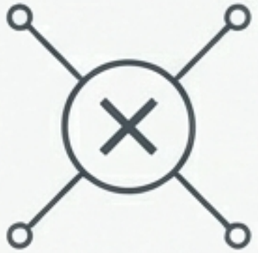
Soma

$O(g+h) \rightarrow O(g) + O(h)$



Produto

$O(g \cdot h) \rightarrow O(g) \cdot O(h)$



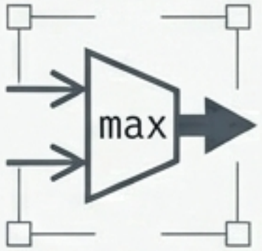
Constantes

$O(k \cdot g) \rightarrow O(g)$



Comportamento Máximo

$f1(n) + f2(n) \rightarrow O(\max \{g(n), h(n)\})$



NotebookLM

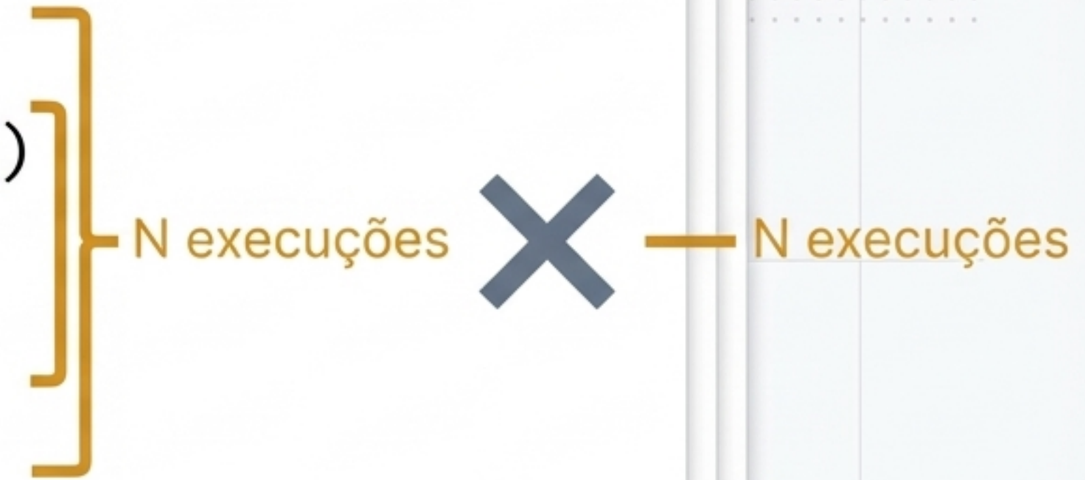
Matriz de Algoritmos Clássicos

Algoritmo	Complexidade
inversão de uma sequência	$O(n)$
cálculo de fatorial	$O(n)$
soma de matrizes	$O(n^2)$
produto de matrizes	$O(n^3)$

NotebookLM

Radiografia de um Código: Loops

```
for(lin=1 ; lin<=N ; lin++){
    for(col=1 ; col<=N ; col++)
        cout<<" - ";
    cout <<endl;
}
```



N × N = $O(n^2)$. Complexidade quadrática típica de varredura em matrizes.

NotebookLM

O Desafio da Recursividade

1

Determinar o número total de chamadas ao procedimento recursivo.

2

Calcular a complexidade de execução de uma única chamada.

Complexidade = (Número de chamadas) × (Complexidade de 1 chamada)

Programas recursivos são mais concisos e têm relação direta com a prova por indução matemática.

NotebookLM

Anatomia de um Fatorial

```
int calcularFatorial(int n) {
  if (n == 0 || n == 1)
    return 1;
  else
    return n * calcularFatorial(n - 1);
}
```

Condição de parada.
Operação elementar -> O(1).

Chamada recursiva.
Reduz o problema em 1 unidade a cada ciclo.

NotebookLM

Desempilhando a Recursão

```
calcularFatorial(4)
→ 4 * calcularFatorial(3)
```

```
calcularFatorial(3)
→ 3 * calcularFatorial(2)
```

```
calcularFatorial(2)
→ 2 * calcularFatorial(1)
```

```
calcularFatorial(1)
→ return 1
```

N chamadas executadas linearmente -> O(n)

NotebookLM

A Relação de Recorrência

Modelando o Fatorial: T(n) = T(n-1) + 1

Passos	iteração
= (T(n-1)) + 1	- 1ª substituição
= (T(n-2) + 1) + 1	
= T(n-2) + 2	- 2ª substituição
= (T(n-3) + 1) + 2	
= T(n-3) + 3	- 3ª substituição
.....	....
= T(n-k) + k	- k-ésima substituição

Quando n-k = 0, T(0) = 1. Assim, n=k, resultando em **T(n) = n + 1 -> O(n)**

NotebookLM

O Teste Final: Raio-X de Polinômios

$n^3 - 1 \rightarrow O(n^3)$

$n^2 + 2.\log n \rightarrow O(n^2)$

$6.547.326 \rightarrow O(1)$

$3.n + 7.m + 2 \rightarrow O(n + m)$

$5.n^2 + 9.m + 4 \rightarrow O(n^2 + m)$

$3.n^n + 5.2^n \rightarrow O(n^n)$

$(n-1)n + n^n - 1 \rightarrow O(n^n)$

$5.3^n + 4.2^{n^2} \rightarrow O(n^{n^2})$

$5.n^7 + 3.2^n + n! \rightarrow O(n!)$

$3.n + 5.m + n.m \rightarrow O(n.m)$

NotebookLM